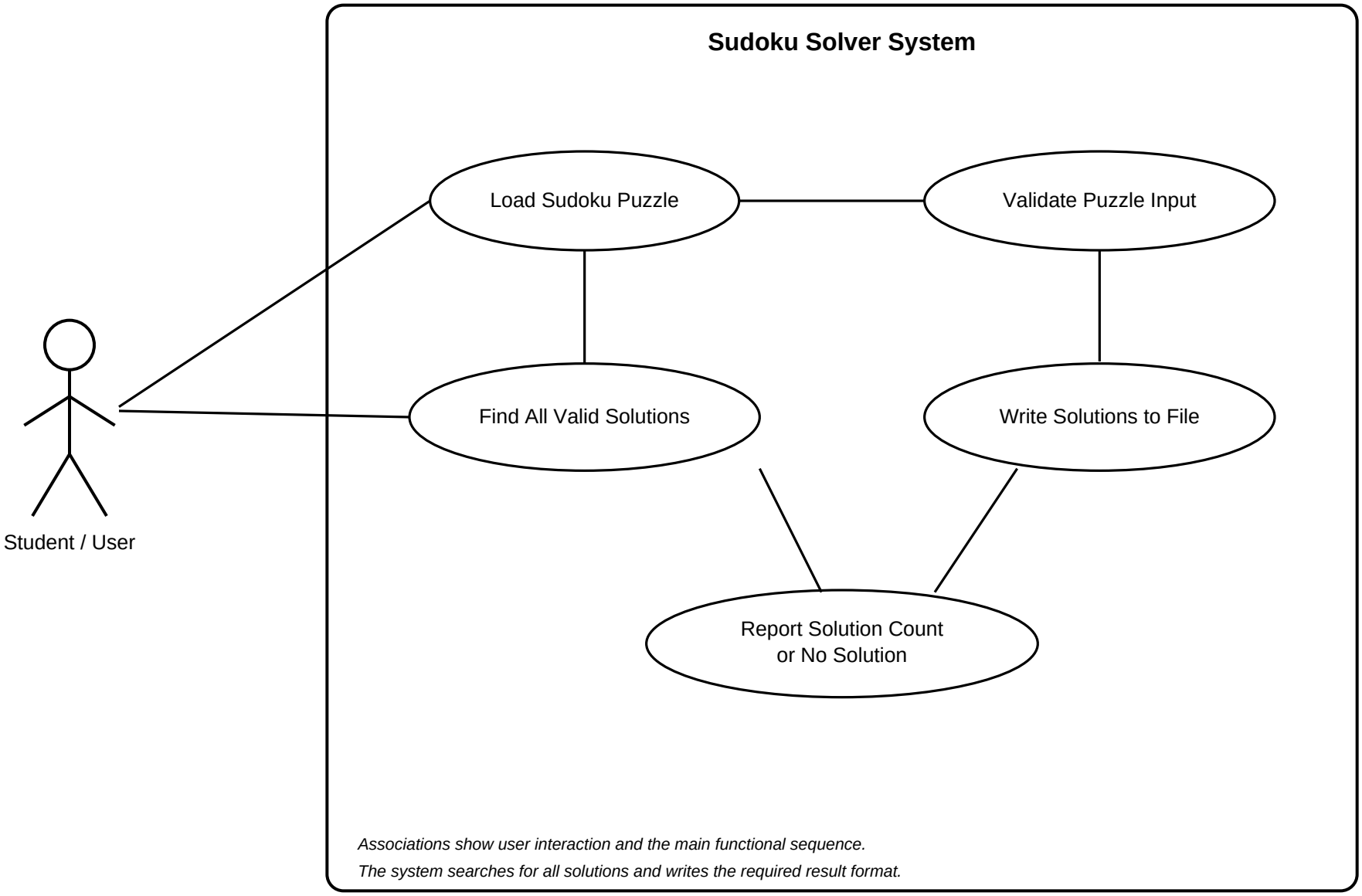
EECS 348 Assignment 5

UML Use Case Diagram — Sudoku Solver



Use Case Documentation

EECS 348 Software Engineering I — Assignment 5
System: Sudoku Solver | Primary actor: Student / User

1. Identify / Discover a Use Case

Use case name: Solve Sudoku Puzzle and Save All Solutions

Primary actor: Student / User running the program.

Goal: Process each supplied Sudoku puzzle, find every valid solution, and save the results in the corresponding solutionN.txt file.

2. Brief Description

The user runs the Sudoku Solver program with puzzle1.txt through puzzle5.txt available in the working directory. The program reads each puzzle, validates its input and starting clues, searches for all valid Sudoku solutions using recursive backtracking, and writes the solutions and final count to the matching output file. If no solution exists, the output file reports “No solution found.”

3. Step-by-Step Flow of Events

1. The user places the five puzzle input files in the program's working directory and runs Assignment5.exe.

2. The program selects the next puzzle file and attempts to open and read its 81 cell values.

3. The program checks that the puzzle uses valid symbols and that its starting clues do not conflict with Sudoku rules.

4. If the puzzle input is invalid or unavailable, the program reports the input error; output-file handling depends on the final implementation.

5. If the puzzle is valid, the solver searches for an empty cell and tries digits 1 through 9 that do not violate row, column, or 3-by-3 block rules.

6. The solver uses recursive backtracking to explore alternatives and continues until every valid completed board has been found.

7. Each completed board is written to the corresponding solutionN.txt file in the same cell format as the input puzzle.

8. After the search, the program writes “Solutions found: n” if one or more solutions exist, or “No solution found.” if none exist.

9. The program repeats the process for the remaining puzzle files and then exits.

4. Detailed Use Case Description

Preconditions:

Assignment5.exe has been compiled for the Cycle server environment, the required puzzle files are available, and the user can read the input files and write output files in the working directory.

Trigger:

The user starts the program.

Main success scenario:

The program reads a puzzle, validates the input and clue consistency, enumerates every valid solution through recursive backtracking, writes each solution to the corresponding output file, and appends the total number of solutions.

Alternative / exception flows:

(a) An input file is missing or cannot be read: the program reports an error and continues according to its implemented error-handling behavior. (b) The puzzle contains invalid symbols or incomplete input: the program rejects the input. (c) The clues contradict Sudoku rules, or the search finds no completion: the output reports “No solution found.” (d) An output file cannot be opened or written: the program reports an output error if that check is implemented.

Postconditions:

For each successfully processed puzzle, the corresponding output file contains all solutions in the required format and a final solution count, or the required no-solution message. The program finishes processing the available puzzle list.